Task 1:

```
struct Student {
   let name: String
   let marks: Int
   let dept: String
}
let students = [
   Student(name: "Asha", marks: 82, dept: "IT"),
   Student(name: "Ravi", marks: 35, dept: "CSE"),
   Student(name: "Meena", marks: 91, dept: "IT"),
   Student(name: "Kiran", marks: 56, dept: "ECE"),
   Student(name: "Divya", marks: 67, dept: "CSE"),
   Student(name: "Arjun", marks: 40, dept: "IT")
]
let passed = students.filter { $0.marks >= 40 }
print("Passed:", passed.map { $0.name })
let names = students.map { $0.name }
print("Names:", names)
let total = students.map { $0.marks }.reduce(0, +)
let average = Double(total) / Double(students.count)
print("Average:", average)
let ranked = students.sorted { $0.marks > $1.marks }
print("Rankings:")
ranked.forEach { print($0.name, $0.marks) }
let grouped = Dictionary(grouping: students, by: { $0.dept })
print("Departments:")
for (dept, list) in grouped {
   print(dept, list.map { $0.name })
}
```

```
let result1 = students.filter { student in
    return student.marks >= 40
}
let result2 = students.filter { $0.marks >= 40 }
print(result1.map { $0.name })
print(result2.map { $0.name })
```

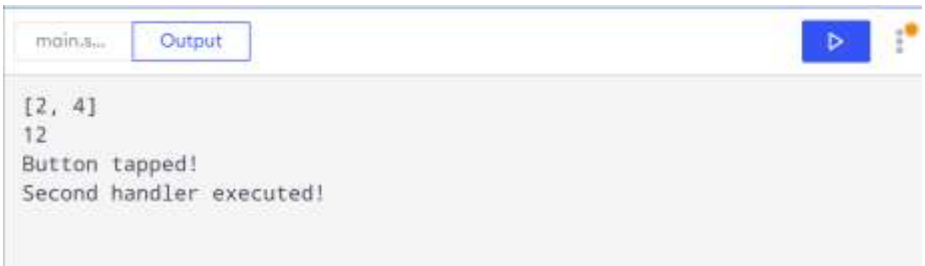
Output:

Task 2:

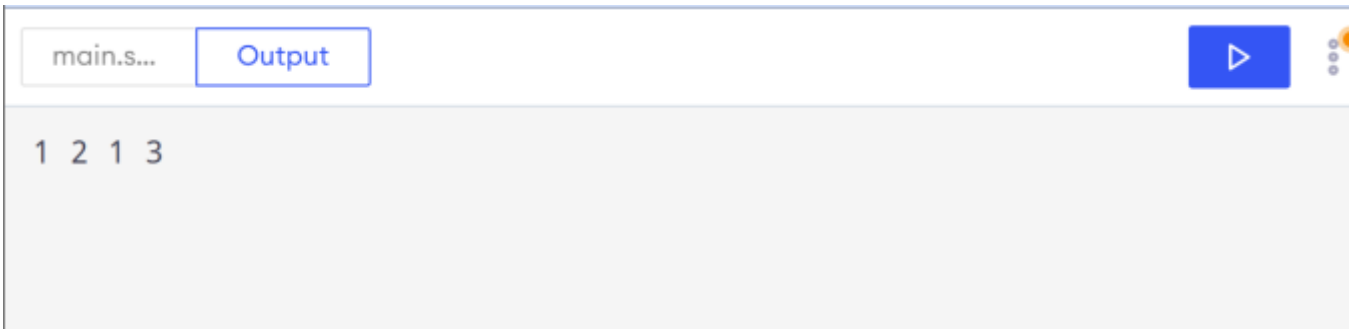
```
func myFilter(_ items: [Int],_ predicate: (Int) -> Bool) -> [Int] {
    var result: [Int] = []
    for item in items {
        if predicate(item) {
            result.append(item)
        }
    }
    return result
}
print(myFilter([1, 2, 3, 4, 5]) { $0 % 2 == 0 })
```

```
func applyTwice( _ value: Int,_ f: (Int) -> Int) -> Int {
   return f(f(value))
}
print(applyTwice(3) { $0 * 2 })
class Button {
   var handlers: [() -> Void] = []
   func onTap(_ handler: @escaping () -> Void) {
      handlers.append(handler)
   }
   func tap() {
      for handler in handlers {
         handler()
      }
   }
}
let button = Button()
button.onTap {
   print("Button tapped!")
}
button.onTap {
   print("Second handler executed!")
}
button.tap()
```

Output:



Task 3:

```
func makeCounter() -> () -> Int {
  var count = 0
  return {
    count += 1
    return count
  }
}
let a = makeCounter()
let b = makeCounter()
print(a(), a(), b(), a())
```

Output: